

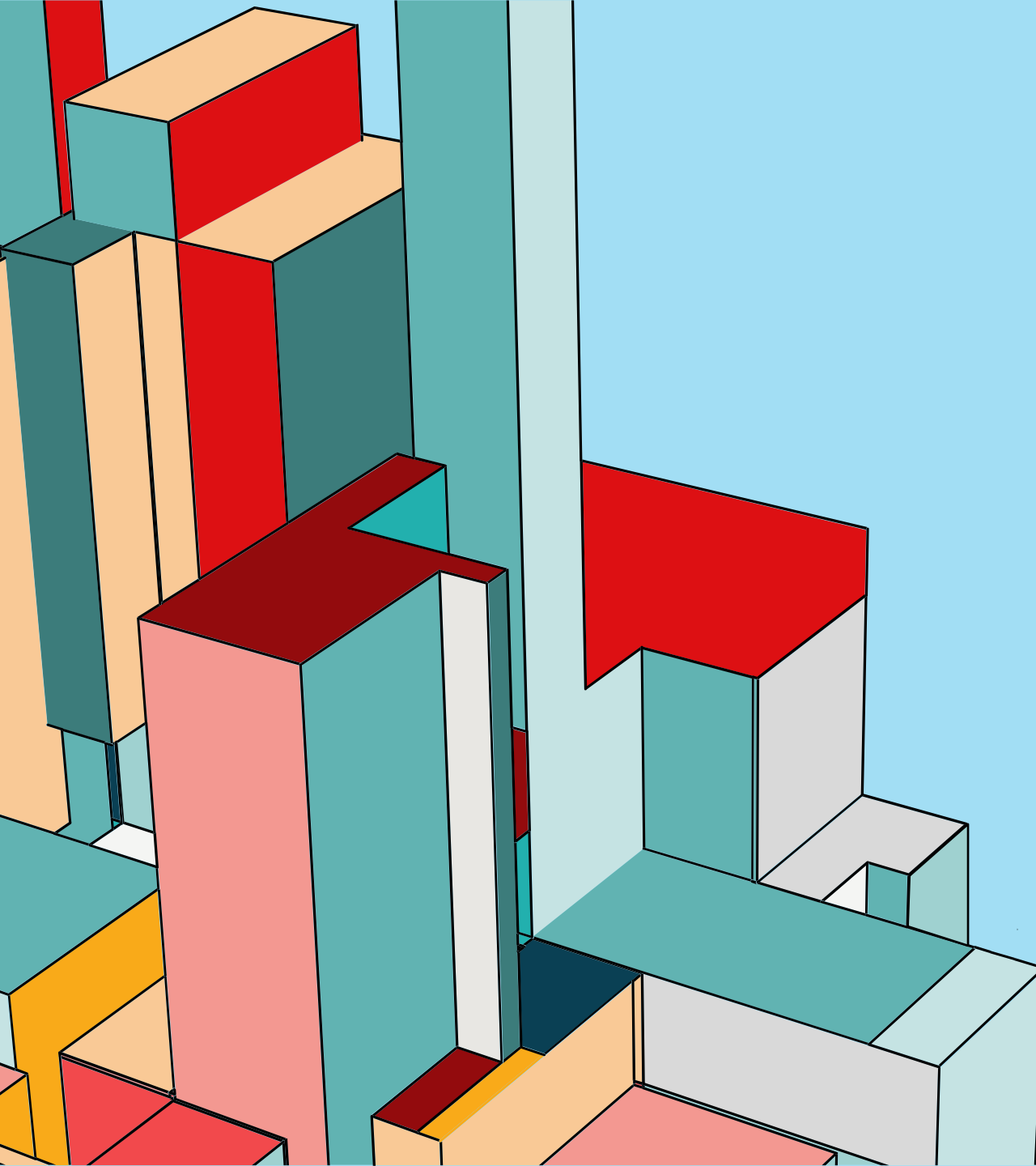
Hey there - I'm Tycho Brahe.
Like me, you might have seen an
ad for NovelAI. It suggests that
you can start writing now,
for free.

Friends, I have
incredible news.

Take control of your story.

Start Writing for Free



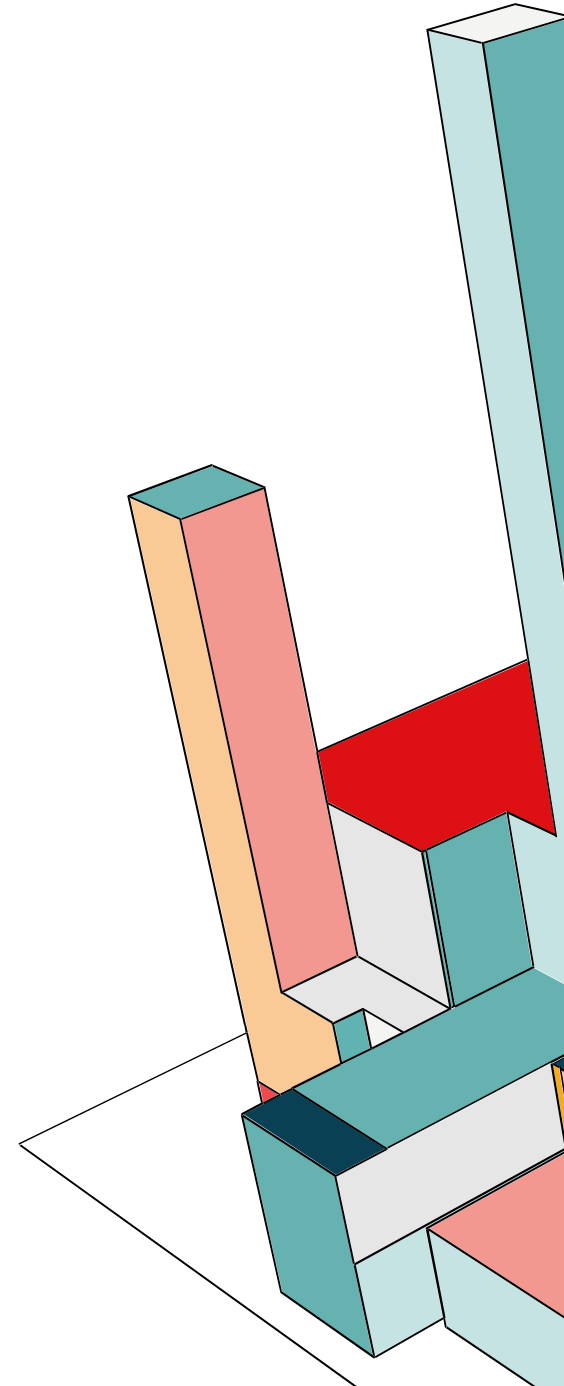


LECTURE 7 **DIVIDE-AND- CONQUER ALGORITHMS II**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS

- Exam 1 is on Wed. 2/11, 8-10am.
 - Open paper notes (your own + slides/HW/etc.)
 - Details Monday
- No labs next week
- HW4 due Tuesday as usual; no new HW next week



TODAY'S TASKS

- Quick review: Writing recurrence relations
- More divide-and-conquer algorithms
 - Quicksort
 - Multiplication of integers
 - Closest pair problem

WRITING RECURRENCE RELATIONS

Given a recursive algorithm:

- Write $T(n)$ = (the time required for a problem of size n)
- Identify each recursive call.
- For each of them, identify how much the problem size decreases and write a term for that recursion. (e.g. $T(n - 1)$, $T(n/2)$, or something similar).
 - When possible, combine like terms. e.g. $T(n - 1) + T(n - 1)$ can be $2T(n - 1)$, but $T(n/2) + T(n/3)$ cannot be combined.
- Finally, add a term for the amount of work done in the non-recursive portion of the algorithm.

WRITING RECURRENCE RELATIONS

An example:

```
# assume integer division throughout
def Kai(A,start,end):
    if end <= start return A[start]
    size = end - start
    w = Kai(A,start,start + size/2)
    x = Kai(A,start + size/4, start + 3*size/4)
    y = Kai(A,start + size/2 + 1, end)
    z = sum(A,start,end)
    return max(w,x,y,z)
```

Write the recurrence relation now.

SUBMITTY POLL #1

What is the recurrence relation for the example algorithm?



WRITING RECURRENCE RELATIONS

An example:

```
# assume integer division throughout
```

```
def Kai(A,start,end):
```

```
    if end <= start return A[start]
```

```
    size = end - start Note: size == n
```

```
    w = Kai(start,start + size/2)
```

```
    x = Kai(start + size/4, start + 3*size/4)
```

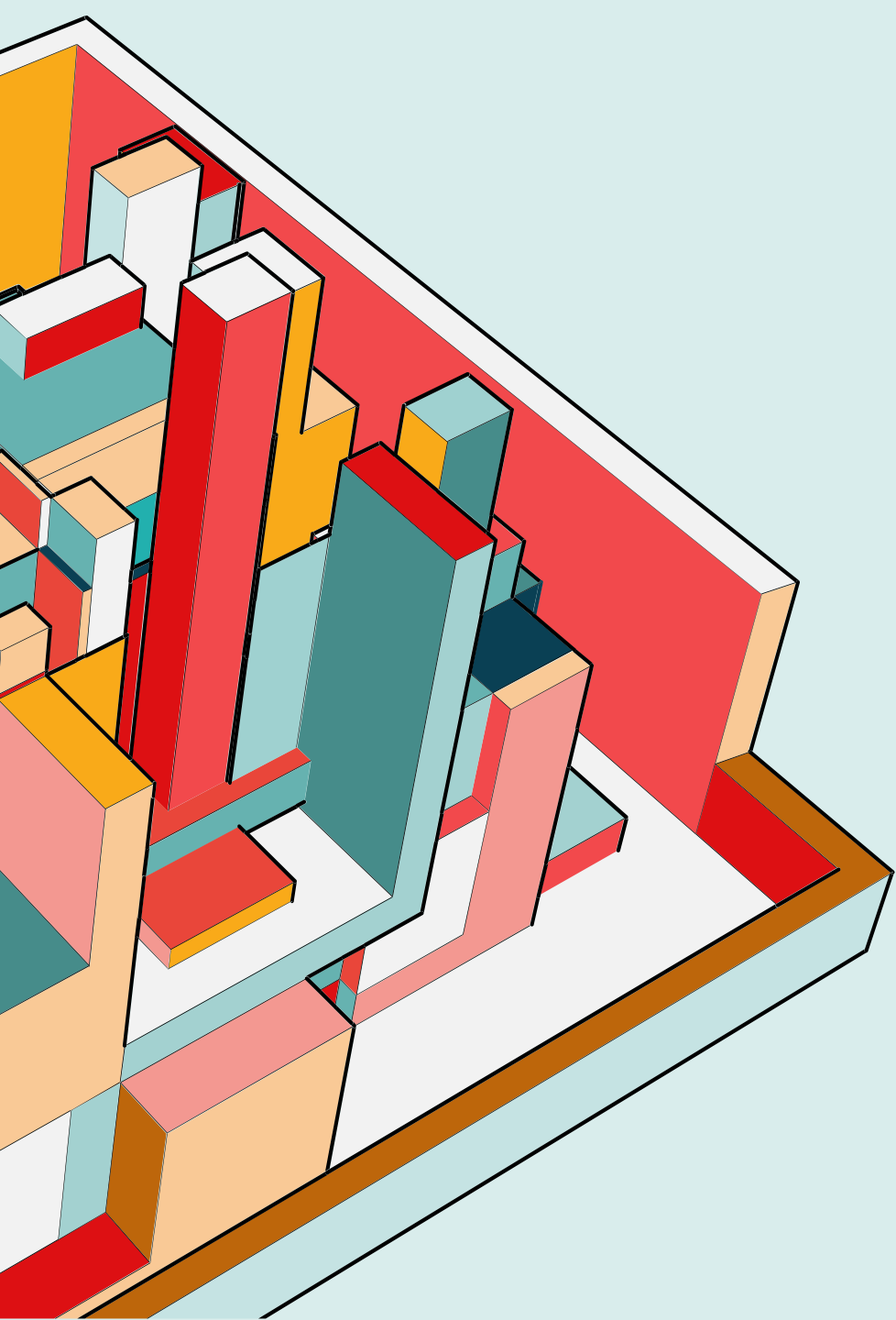
```
    y = Kai(start + size/2 + 1, end)
```

```
    z = sum(A,start,end)  $\Theta(n)$ 
```

```
    return max(w,x,y,z)  $\Theta(1)$ 
```



Thus, the recurrence is $T(n) = 3 \cdot T(n/2) + \Theta(n)$



QUICKSORT

PARTITIONING II: ELECTRIC BOOGALOO

Our initial concept of partitioning was as a partial sort.

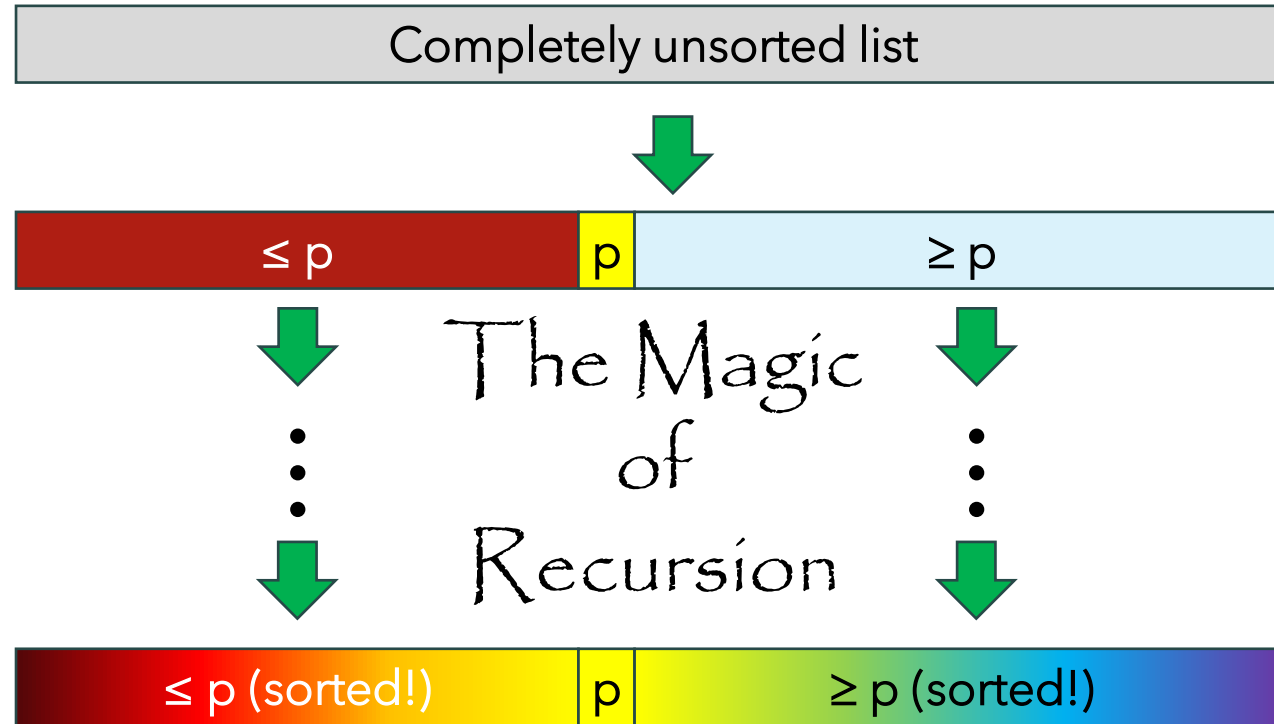


We then used this recursively to find the k -th smallest element of the list.

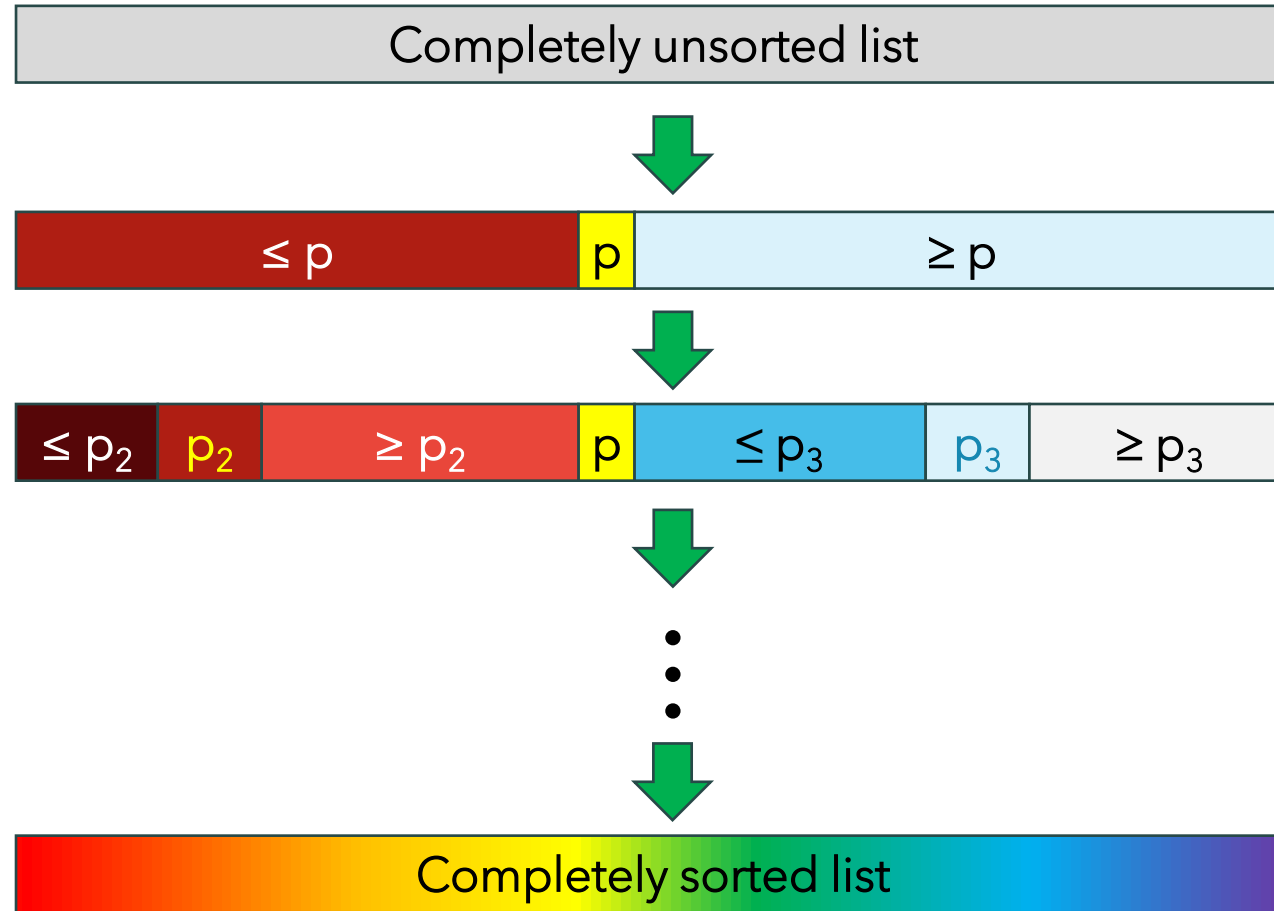
How might we use the same idea to fully sort the list?

- Remember: Quickselect operates by ignoring the half of the partition that we don't need to look in.
- But this lecture is about divide-and-conquer...
- What if we recursively partition both halves of the list?

QUICKSORT IN PICTURES



QUICKSORT IN PICTURES



QUICKSORT PSEUDOCODE

```
def quicksort(A,L,R):  
    if L < R:  
        s = partition(A,L,R)  
        quicksort(A,L,s-1)  
        quicksort(A,s+1,R)
```

CONTRASTING MERGESORT AND QUICKSORT

Mergesort

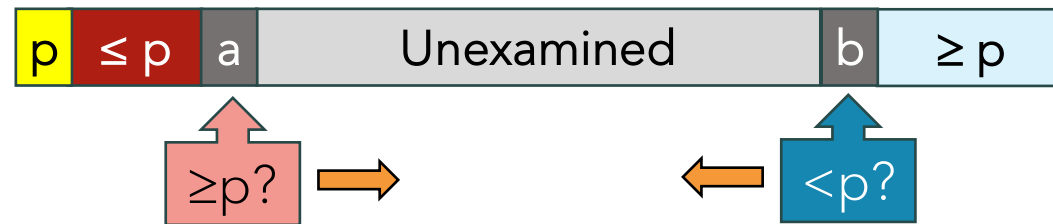
- Divides items based on array index
- Splitting is trivial; work is done during merge
- Guarantees good splits
- Requires extra memory

Quicksort

- Divides items based on their value
- Partition does all the work; recombining takes zero effort
- Good splits depend on good pivots
- Sorts in-place* (needs $\log n$ space in the stack)

HOARE'S PARTITION ALGORITHM

Quicksort was invented by Tony Hoare in 1960. His approach to partitioning is slightly more complex, but results in fewer overall swaps. The key idea is that we work from both sides of the array.



We scan from left to right until we find a "big" element, and from right to left until we find a "small" element. Then we swap them.

HOARE PARTITIONING PSEUDOCODE

```
def HoarePartition(A,L,R):  
    p = A[L]  # again, we could be more clever  
    i = L+1; j = R  
    while i < j:  
        while A[i] < p: i++  # need to avoid index OOB  
        while A[j] > p: j--  
        swap(A,i,j)  
    swap(A,i,j)  # undo last swap; doing it this way  
                # saves an extra if inside the loop  
    swap(A,L,j)  
    return j
```

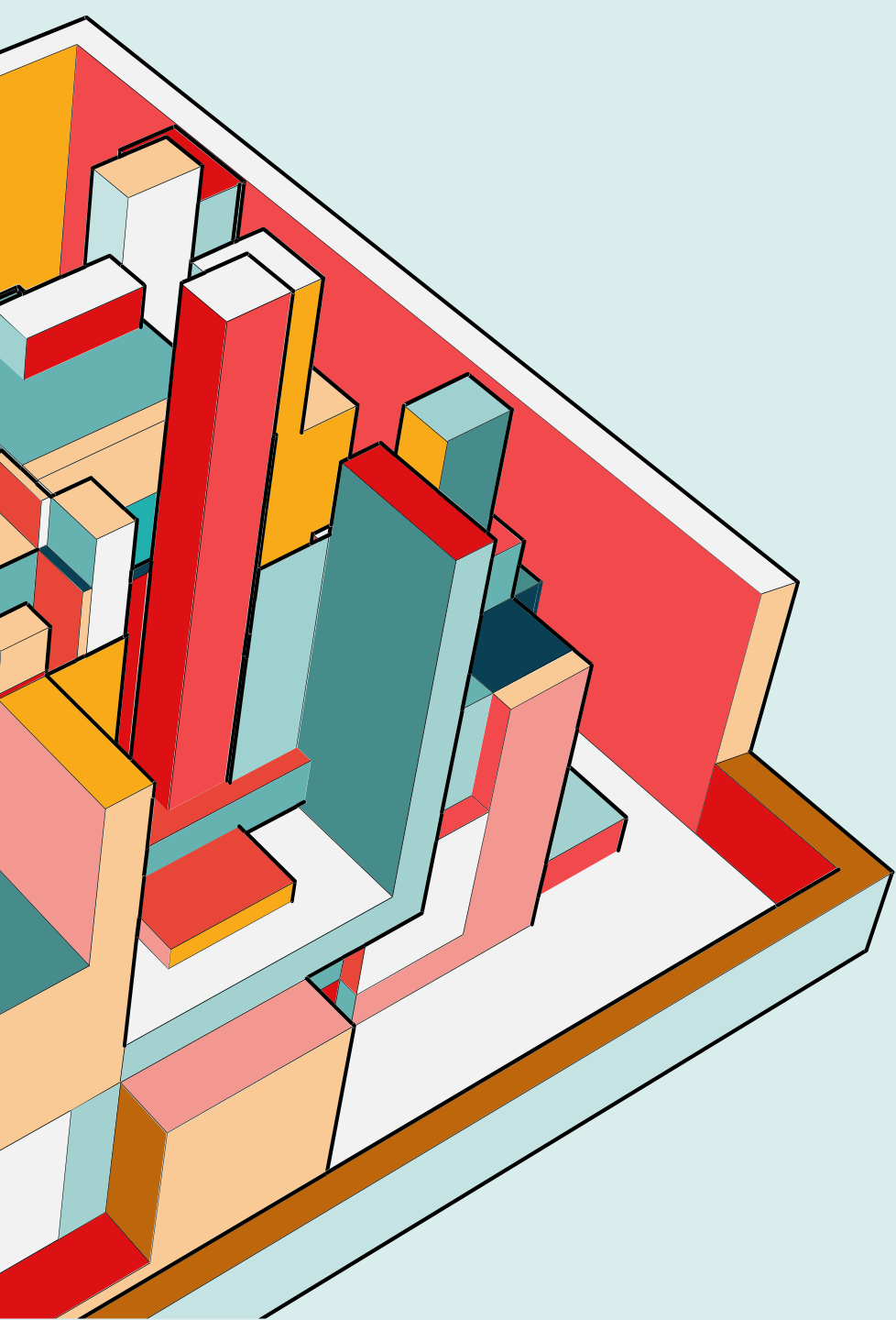
QUICKSORT: RUNTIME ANALYSIS

Since it operates off of the same mechanism as Quickselect, it should be unsurprising that Quicksort shares the same best, worst, and average cases.

- Best case: Every partition perfectly splits the list in half. We get $Q(n) = 2Q(n/2) + n$, which means $Q(n) = \Theta(n \log n)$. (In fact, it's exact: $Q(n) = n \lg n$.)
- Worst case: Every partition splits off only a single element. (e.g. Sorted list if we always use the first element as a pivot.) We get $Q(n) = Q(n - 1) + n$, which gives $Q(n) = \Theta(n^2)$.
- Average case: Depends on the data distribution and the pivot selection. Assuming uniform random data and naïve pivots, $Q(n) \approx 1.39n \lg n$, which is obviously $\Theta(n \log n)$.

INCREMENTAL IMPROVEMENTS TO QUICKSORT

- One can try for better pivots. A randomly-selected pivot avoids trouble in nearly-sorted lists. The *median-of-three* approach (middle value out of leftmost, center, and rightmost) is very common – it provides significant safety for very little computational overhead.
- In lists with lots of duplicates, one can use a modified partition that breaks things into $< p$, $> p$, and $== p$.
- When the list is small enough, we can switch to insertion sort. (Or even not sort small lists at all and then apply an insertion sort pass at the end.)



OTHER DIVIDE-AND- CONQUER ALGORITHMS

MULTIPLICATION

What is 34×27 ?

Observe that the standard “pencil-and-paper” algorithm uses the distributive property – we treat 34×27 as $34 \times (20 + 7) = 34 \times 7 + 34 \times 20$.

And those multiplications are broken down further: $34 \times 7 = (30 + 4) \times 7$ and $34 \times 20 = (30 + 4) \times 20$.

The factors of 10 don't really add work:

$$34 \times 27 = (3 \cdot 10^1 + 4 \cdot 10^0) \times (2 \cdot 10^1 + 7 \cdot 10^0) = (3 \times 2)10^2 + (3 \times 7 + 4 \times 2)10^1 + (4 \times 7)10^0$$

$$\begin{array}{r} 34 \\ \times 27 \\ \hline 238 \\ 680 \\ \hline 918 \end{array}$$

MULTIPLICATION

$$34 \times 27 = (3 \cdot 10^1 + 4 \cdot 10^0) \times (2 \cdot 10^1 + 7 \cdot 10^0) = \\ (3 \times 2)10^2 + (3 \times 7 + 4 \times 2)10^1 + (4 \times 7)10^0$$

Observe that this formulation has the same four single-digit multiplications that the pencil-and-paper approach uses. Can we reduce this?

The trick here is to use the fact that we must compute 3×2 and 4×7 to reduce the number of additional multiplications we need to do to calculate the middle term.

$$\begin{array}{r} 34 \\ \times 27 \\ \hline 238 \\ 680 \\ \hline 918 \end{array}$$

MULTIPLICATION

$$\text{34} \times \text{27} = (3 \cdot 10^1 + 4 \cdot 10^0) \times (2 \cdot 10^1 + 7 \cdot 10^0) =$$

$$(\text{3} \times \text{2})10^2 + (\text{3} \times \text{7} + \text{4} \times \text{2})10^1 + (\text{4} \times \text{7})10^0$$

$$(\text{3} \times \text{7} + \text{4} \times \text{2}) = 29; \text{3} \times \text{2} = 6; \text{4} \times \text{7} = 28$$

Observe that $29 + 6 + 28 = 63$, which is 7×9 ...

... which is $(\text{3} + \text{4}) \times (\text{2} + \text{7})$!

In other words, we can obtain $(\text{3} \times \text{7} + \text{4} \times \text{2})$ by instead calculating $(\text{3} + \text{4}) \times (\text{2} + \text{7}) - (\text{3} \times \text{2}) - (\text{4} \times \text{7})$.

Since we'd already done the latter two multiplications, this means the full problem can be done with only three total single-digit multiplications.

$$\begin{array}{r} \text{3 4} \\ \times \text{2 7} \\ \hline \text{2 3 8} \\ \text{6 8 0} \\ \hline \text{9 1 8} \end{array}$$

GENERALIZING

Of course, the process can be used on any pair of two-digit numbers, not just 34×27 .

Let $a = 10a_1 + a_0$ and $b = 10b_1 + b_0$ (so that a_1, a_0 are the individual digits of a , and similarly for b).

To calculate $c = a \times b$, we can compute:

$$c_2 = a_1 \times b_1$$

$$c_0 = a_0 \times b_0$$

$$c_1 = (a_1 + a_0) \times (b_1 + b_0) - c_2 - c_0$$

$$c = 10^2 c_2 + 10^1 c_1 + 10^0 c_0$$

SCALING UP

So far, a reaction of “so what?” here is very reasonable – single-digit multiplications are about the same amount of computation as single-digit additions.

But what about larger numbers?

Let a and b be n -digit values (for simplicity, assume n is even).

Use the same technique to show how you can calculate $c = a \times b$ using only three multiplications of $n/2$ -digit values.

SCALING UP

Let a and b be n -digit values (for simplicity, assume n is even).

Write $a = a_1 \cdot 10^{n/2} + a_0$ and $b = b_1 \cdot 10^{n/2} + b_0$.

Now we can write c in similar fashion as before:

$$c_2 = a_1 \times b_1$$

$$c_0 = a_0 \times b_0$$

$$c_1 = (a_1 + a_0) \times (b_1 + b_0) - c_2 - c_0$$

$$c = c_2 \cdot 10^n + c_1 \cdot 10^{n/2} + c_0$$

Write the recurrence relation governing this process.

SUBMITTY POLL #2

What is the recurrence relation for the multiplication algorithm?



SCALING UP - RECURRENCE

Let a and b be n -digit values (for simplicity, assume n is even).

Write $a = a_1 \cdot 10^{n/2} + a_0$ and $b = b_1 \cdot 10^{n/2} + b_0$.

Now we can write c in similar fashion as before:

$$c_2 = a_1 \times b_1$$

$$c_0 = a_0 \times b_0$$

$$c_1 = (a_1 + a_0) \times (b_1 + b_0) - c_2 - c_0$$

$$c = c_2 \cdot 10^n + c_1 \cdot 10^{n/2} + c_0$$

We have three multiplications of half the size. We also have six additions/subtractions, which take $O(n)$ time. So the recurrence is $T(n) = 3T(n/2) + O(n)$.

REVISITING CLOSEST PAIR

Recall the Closest Pair problem: Given a collection of points in the plane, find the pair of points that is closest together.

We explored a brute force approach that looked at every pair of points, resulting in an $O(n^2)$ algorithm.

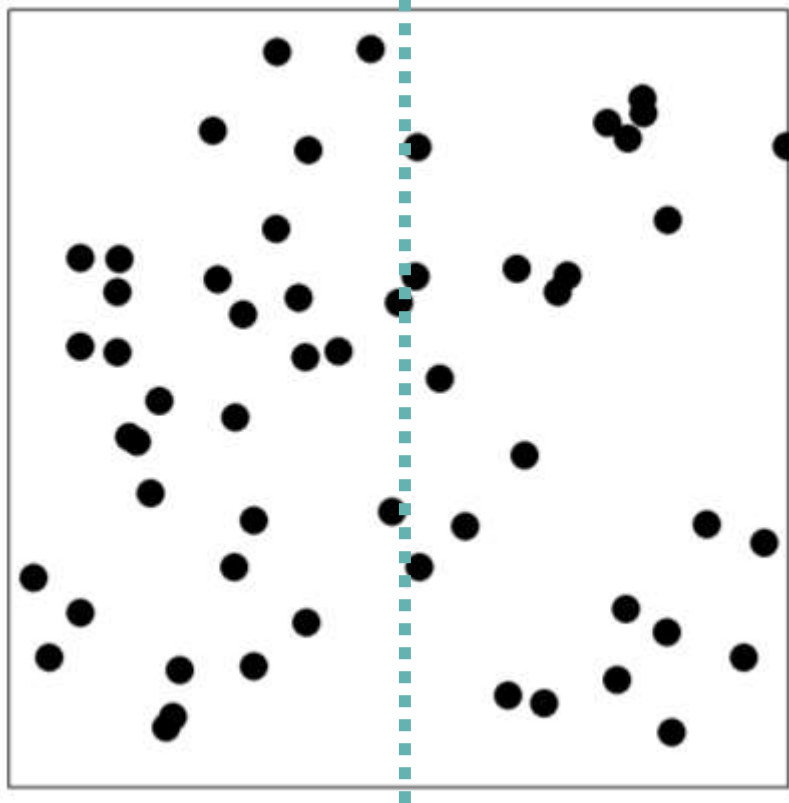
Divide-and-conquer can get us a faster algorithm.

The process begins by sorting all of the points according to their x coordinate (could use y instead), so it will clearly be $\Omega(n \log n)$ time.

Try to devise such an approach now.

REVISITING CLOSEST PAIR

Given a list sorted by x coordinate, we select a line $x = m$ that puts half the points on either side, and then recursively call our algorithm to find the closest pair in each half.



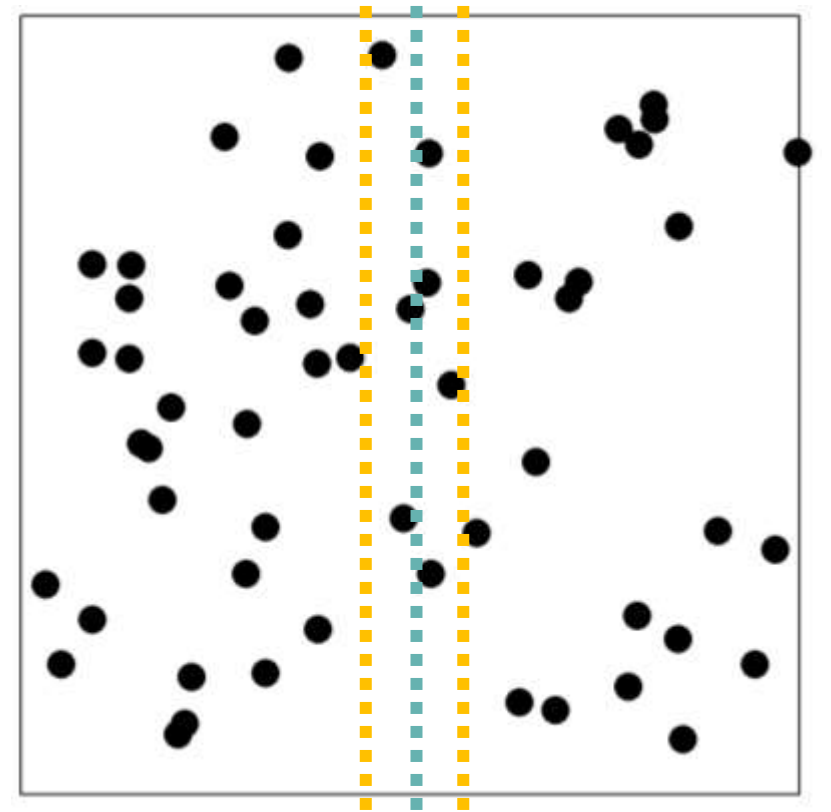
REVISITING CLOSEST PAIR

Once we have the answers d_1 and d_2 from each half, obviously we look at the smaller of those, $d = \min(d_1, d_2)$.

But what if the closest pair crosses the midline?

Well, it must be no further than d from that midline.

And it turns out that there can be no more than $O(n)$ pairs of points that are within d of the midline, so we're good.



BEFORE YOU LEAVE

